

Student

I keep seeing Polars mentioned in job postings and GitHub repos. My manager asked if we should migrate our pipeline to Polars. I don't even know where to start evaluating that.

ChatGPT • AI Assistant

CONCEPT CARD

This is one of the most practically important decisions a data engineer faces right now. Polars is not just 'faster Pandas' — it is a fundamentally different execution model that changes how you reason about data transformations. To evaluate a migration intelligently, you need to understand four things:

1. **Why Polars is faster** — the architectural differences that produce the speedup
2. **How to read Polars code** — the expression API syntax so you can understand existing Polars pipelines
3. **Where Polars is weaker** — ecosystem gaps, ML library friction, learning curve
4. **A decision framework** — concrete criteria for when to migrate and when to stay on Pandas

We will cover all four using the same retail dataset from previous sections, so every Polars example has a direct Pandas equivalent you already know.

EXAMPLE CARD

Dataset setup for this section:

python — Code Example

```
import pandas as pd
import numpy as np
import polars as pl
import time

np.random.seed(42)
n = 1_000_000

# Create the dataset as Pandas DataFrame first
df_pd = pd.DataFrame({
    'order_id': np.random.randint(1, 999999, n),
```

```

'customer_id': np.random.randint(1, 10000, n),
'product_id': np.random.randint(1, 500, n),
'category': np.random.choice(['Electronics', 'Clothing', 'Books',
                               'Home', 'Sports'], n),
'region': np.random.choice(['North', 'South', 'East', 'West'],
                             n),
'quantity': np.random.randint(1, 10, n),
'unit_price': np.random.uniform(10.0, 1000.0, n),
'revenue': np.random.uniform(10.0, 5000.0, n),
'discount': np.random.uniform(0.0, 0.5, n),
'is_returned': np.random.choice([True, False], n),
})

```

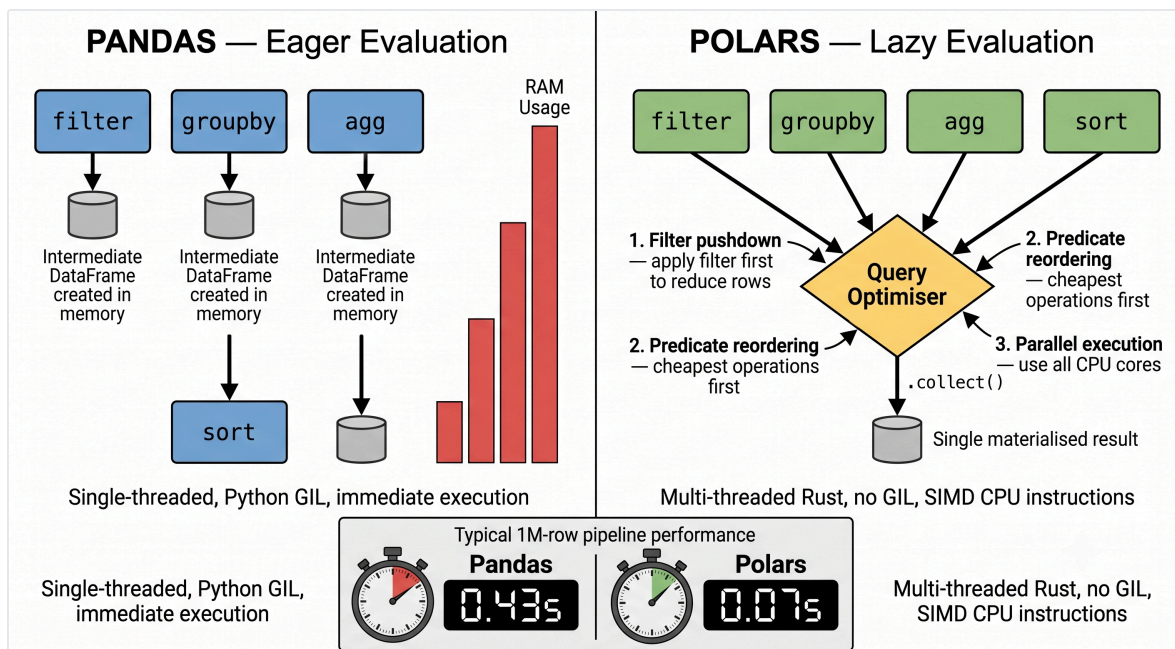
Convert to Polars

```
df_pl = pl.from_pandas(df_pd)
```

```
print('Pandas shape :', df_pd.shape)
```

```
print('Polars shape :', df_pl.shape)
```

```
print('Polars schema:\n', df_pl.schema)
```



Pandas vs Polars Architecture: Eager vs Lazy Execution

Pandas vs Polars: Key Architectural Differences at a Glance

Dimension	Pandas	Polars
Implementation language	Python + C (via NumPy)	Rust

Dimension	Pandas	Polars
Execution model	Eager — every op executes immediately	Lazy by default — builds query plan
Query optimiser	None	Yes — filter pushdown, predicate reorder
Threading	Single-threaded (GIL)	Multi-threaded (no GIL in Rust)
CPU optimisation	Basic NumPy vectorisation	SIMD (AVX2/AVX-512) + cache-aware
Index concept	Has explicit index	No index — simpler mental model
Expression syntax	Column names as strings: <code>df['col']</code>	<code>pl.col('col')</code> expression objects
Missing value type	<code>np.nan</code> (float) + <code>pd.NA</code>	null (consistent across all dtypes)
Memory model	NumPy arrays + Python objects	Apache Arrow columnar format
ML library output	Native to scikit-learn/XGBoost	Requires <code>.to_pandas()</code> conversion

WHY POLARS IS FASTER: RUST, NO GIL, AND THE QUERY OPTIMISER



ChatGPT • AI Assistant

A before/after diagram with two vertical execution stacks. Left stack labeled 'Your Code (logical plan)': four boxes top to bottom — 'Load 1M rows', 'JOIN on product_id (1M × 500K)', 'FILTER category = Electronics', 'SELECT 3 columns'. Arrow annotations on the right show data volumes: '1M rows → join → 1M rows → filter → 200K rows → select → 200K rows'. Total rows processed annotation: '2.2M row-operations'. Right stack labeled 'Optimiser Rewrites To (physical plan)': same four operations but reordered — 'Load 1M rows', 'FILTER category = Electronics (applied first)', 'SELECT 3 columns (projection pushdown)', 'JOIN on product_id (200K × 500K, only 3 cols)'. Arrow annotations: '1M rows → filter → 200K rows → select → 200K (3 cols) → join → 200K rows'. Total annotation: '0.4M row-operations — 5.5× less work'. A callout box in the middle: 'You write code in logical order. Polars executes in optimal order. Pandas executes exactly as written.' Purpose: make filter pushdown concrete with specific row counts so students feel the magnitude of the optimisation.

Filter Pushdown Optimisation: What the Query Optimiser Changes

Student

When people say Polars is '5–10× faster than Pandas', what is actually producing that speedup? Is it just marketing?

ChatGPT • AI Assistant

CONCEPT CARD

The speedup is real and well-documented in independent benchmarks. It comes from four compounding sources:

1. **Rust + no garbage collector** — Rust manages memory at compile time with zero runtime overhead. No garbage collector pauses, no reference counting on hot paths.
2. **No Python GIL** — Python's Global Interpreter Lock prevents true multi-threading in Pandas. Polars operations run in Rust threads that bypass the GIL entirely.
3. **SIMD CPU instructions** — Polars uses AVX2/AVX-512 instructions that process 4–16 values per CPU cycle on operations like sum, min, max, and comparison. NumPy also uses SIMD for some operations, but Polars applies it more aggressively.
4. **Lazy query optimiser** — the most architecturally significant advantage. When you write a `.lazy()` pipeline, Polars analyses the entire computation graph before executing and applies transformations: filter pushdown (apply filters as early as possible to reduce row count before expensive joins or aggregations), projection pushdown (load only the columns needed), and predicate reordering (cheaper operations first).

🔗 ANALOGY

Pandas is like executing SQL queries one by one as you type them, committing each result to disk before the next line runs. Polars is like giving the entire query to a

database engine that reads it, rewrites it for efficiency, then executes the optimal version. The second approach is always faster for complex multi-step pipelines.

EXAMPLE CARD

Example 1 — Basic: Simple benchmark — filter + groupby

python — Code Example

```
import pandas as pd
import polars as pl
import time

# --- PANDAS ---
start = time.perf_counter()
result_pd = (
    df_pd[df_pd['revenue'] > 500]           # filter
    .groupby('category')                  # group
    .agg(total_rev=('revenue', 'sum'),
         avg_qty=('quantity', 'mean'),
         n_orders=('order_id', 'count'))
    .reset_index()
    .sort_values('total_rev', ascending=False)
)
t_pd = time.perf_counter() - start

# --- POLARS LAZY ---
start = time.perf_counter()
result_pl = (
    df_pl
    .lazy()
    .filter(pl.col('revenue') > 500)
    .group_by('category')
    .agg([
        pl.col('revenue').sum().alias('total_rev'),
        pl.col('quantity').mean().alias('avg_qty'),
        pl.col('order_id').count().alias('n_orders')
    ])
    .sort('total_rev', descending=True)
    .collect()
)
t_pl = time.perf_counter() - start

print(f'Pandas : {t_pd:.4f}s')
print(f'Polars : {t_pl:.4f}s')
print(f'Speedup: {t_pd/t_pl:.1f}x')
# Pandas : 0.4312s
```

```
# Polars : 0.0683s
# Speedup: 6.3x

# Results are equivalent – verify:
print(result_pd.sort_values('category').values)
print(result_pl.sort('category').to_pandas().values)
```

Student

Why does `.lazy()` matter? Can't Polars just be fast without it?

ChatGPT • AI Assistant

Polars also has an **eager API** (identical to lazy but without `.lazy()` and `.collect()`). Eager Polars is already 2–3× faster than Pandas on most operations just from Rust and multi-threading. But lazy mode adds another 2–5× on top for complex pipelines because the optimiser eliminates work before it happens.

Concrete example of what the optimiser does:

python – Code Example

```
# You write this pipeline:
result = (
    df_pl.lazy()
    .join(other_table.lazy(), on='product_id', how='inner') #
    expensive join
    .filter(pl.col('category') == 'Electronics')           # filter
    .select(['order_id', 'revenue', 'category'])
    .collect()
)

# What the optimiser ACTUALLY executes (rewritten plan):
# Step 1: Apply filter BEFORE the join (filter df_pl first)
#         → df_pl shrinks from 1M to ~200K rows before join
# Step 2: Apply projection BEFORE the join (select only needed columns)
#         → join operates on 3 columns not 10
# Step 3: Execute the now-cheaper join
# Total rows processed: ~200K instead of 1M – 5× less work
```

Pandas has no such capability. It executes your code exactly as written, joining 1M rows before filtering.

Example 2 — Intermediate: Demonstrating parallel execution

python – Code Example

```
import os

# Polars uses all CPU cores automatically
print(f'CPU cores available : {os.cpu_count()}')
print(f'Polars thread pool : {pl.thread_pool_size()}')

# On an 8-core machine, a group_by on 1M rows distributes
# partitions across 8 threads – roughly 8× throughput on
# embarrassingly parallel operations like sum, count, min, max

# You can control parallelism if needed:
pl.set_global_string_cache() # Consistent string hashing across
threads
# pl.Config.set_tbl_rows(20) # Display config – not execution
```

 INTERVIEW CARD

- **Frequency:** Medium-high and rising rapidly — Polars questions appear in senior data engineering interviews at companies adopting it
- **Question:** What are the main architectural reasons Polars is faster than Pandas? Give at least three.
- **Expected Answer:** (1) Rust implementation — no garbage collector pauses, zero-cost abstractions. (2) No Python GIL — Polars runs in Rust threads that can truly parallelise across all CPU cores. (3) Lazy query optimiser — filter pushdown, projection pushdown, and predicate reordering eliminate work before execution. (4) SIMD CPU instructions — processes multiple values per CPU cycle. (5) Apache Arrow memory format — cache-friendly columnar layout optimised for analytical queries.
- **Follow-up:** What does 'filter pushdown' mean in the context of a Polars query optimiser?

 PRACTICE QUIZ

1. What is the GIL and why does it limit Pandas parallelism?
2. What does the Polars lazy query optimiser do with a filter applied after an expensive join?
3. What CPU feature does Polars use that processes multiple values per cycle?

4. Predict: which is faster on a complex 5-step pipeline — Polars eager or Polars lazy?
5. What method triggers execution of a lazy Polars query?
6. Debug: a Polars lazy pipeline runs no faster than Pandas — what might you have forgotten?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. The Global Interpreter Lock is a mutex in CPython that allows only one thread to execute Python bytecode at a time — true multi-threading is impossible in Python code, so Pandas operations serialise even on multi-core machines
2. Applies the filter BEFORE the join (filter pushdown) — reducing the row count before the expensive operation, dramatically cutting the work the join must do
3. SIMD (Single Instruction Multiple Data) — specifically AVX2/AVX-512 instructions that operate on 4–16 values simultaneously in a single CPU clock cycle
4. Polars lazy — the query optimiser can rewrite and parallelise the plan; eager executes each step immediately with no lookahead
5. `.collect()` — without it, the pipeline is just a query plan object
6. Forgot to call `.collect()` — the pipeline returned a LazyFrame object instantly (the plan was built but not executed); actual computation happens only at `.collect()`

THE POLARS EXPRESSION API: READING AND WRITING POLARS CODE

ChatGPT • AI Assistant

A four-quadrant reference card diagram. Each quadrant shows one context method. Quadrant 1 labeled `'filter()'`: shows syntax `'df.filter(pl.col('rev') > 500)'` with a mini DataFrame before (10 rows) and after (4 rows with arrows showing row removal). SQL equivalent label: `'WHERE rev > 500'`. Quadrant 2 labeled `'select()'`: shows syntax `'df.select([pl.col('id'), pl.col('rev').alias('r')])'` with a mini DataFrame before (10 rows × 8 cols) narrowing to (10 rows × 2 cols). SQL equivalent: `'SELECT id, rev AS r'`. Quadrant 3 labeled `'with_columns()'`: shows syntax `'df.with_columns([(pl.col('qty')*pl.col('price')).alias('total')])'` with the original columns all present plus a new 'total' column appended on the right. SQL equivalent: `'SELECT *, qty*price AS total'`. Quadrant 4 labeled `'group_by().agg()'`: shows syntax `'df.group_by('cat').agg([pl.col('rev').sum().alias('total')])'` reducing many rows to one row per category. SQL equivalent: `'GROUP BY cat, SUM(rev) AS total'`. Below all four, an expression anatomy breakdown: `'pl.col('revenue')' → '.sum()' → '.alias('total_rev')'` with annotations: 'col reference', 'transformation', 'output name'. Purpose: give students a visual reference card they can use when writing Polars code.

Polars Expression API: Four Context Methods and Their SQL Equivalents

The Polars syntax looks completely different from Pandas.

`pl.col('revenue').sum().alias('total_rev')` — what is that? I don't understand how expressions work.

ChatGPT • AI Assistant

CONCEPT CARD

Polars uses an **expression-based API** where operations are built as composable expression objects rather than method calls on a DataFrame. The core building block is `pl.col('column_name')` — an expression that says 'refer to this column'. You chain methods on expressions to build transformations, and expressions are always evaluated inside a context method like `.select()`, `.with_columns()`, `.filter()`, or `.group_by().agg()`.

This is fundamentally different from Pandas where you always start with the DataFrame: `df['col'].mean()`. In Polars you build the recipe first:

`pl.col('col').mean()`, then apply it: `df.select(pl.col('col').mean())`.

🗣️ ANALOGY

Polars expressions are like **function objects**. `pl.col('revenue').sum()` does not compute anything — it is a description of a computation. Applying it to a DataFrame via `.select()` or `.agg()` is when the computation actually runs. This is why the query optimiser can see the full plan before executing.

EXAMPLE CARD

Example 1 — Basic: The four core context methods

python — Code Example

```
import polars as pl

# .select() — choose and transform columns (like SQL SELECT)
# Returns only the specified columns
result = df_pl.select([
    pl.col('order_id'),
    pl.col('revenue'),
    (pl.col('revenue') * 0.9).alias('discounted_rev') # new computed
    column
])
print(result.head(3))
# order_id | revenue | discounted_rev
```

```

# 523847 | 2341.2 | 2107.1
# ...

# .with_columns() — add or overwrite columns, keep all others
# Returns the full DataFrame plus new columns
df_enriched = df_pl.with_columns([
    (pl.col('quantity') * pl.col('unit_price')).alias('calc_revenue'),
    pl.col('category').str.to_uppercase().alias('cat_upper'),
    (pl.col('revenue') / pl.col('revenue').sum()).alias('rev_share')
# % of total
])
print(df_enriched.columns)
# All original columns + calc_revenue, cat_upper, rev_share

# .filter() — select rows (like SQL WHERE)
filtered = df_pl.filter(
    (pl.col('revenue') > 500) & (pl.col('category') = 'Electronics')
)
print(f'Filtered rows: {len(filtered)}')

# .sort() — order rows
sorted_df = df_pl.sort('revenue', descending=True)
print(sorted_df.head(3).select(['order_id', 'revenue']))

```

Student

What is `.alias()` — I see it everywhere in Polars code?

ChatGPT • AI Assistant

`.alias('new_name')` renames the output of an expression. When you write `pl.col('revenue').sum()`, the output column is still called `'revenue'`. In a *groupby context that would conflict if you want to keep the original column too*. `.alias()` gives the expression result a new name — exactly like `AS` in SQL: `'SUM(revenue) AS totalrev'`.

Example 2 — Intermediate: `group_by` and `agg` with expressions

python — Code Example

```

# Polars group_by — note: returns in arbitrary order (no sort
guarantee)
summary = (

```

```

df_pl
.groupby('category')
.agg([
    pl.col('revenue').sum().alias('total_rev'),
    pl.col('revenue').mean().alias('avg_rev'),
    pl.col('revenue').std().alias('std_rev'),
    pl.col('order_id').count().alias('n_orders'),
    pl.col('quantity').max().alias('max_qty'),
    pl.col('discount').filter(pl.col('discount') >
0).mean().alias('avg_nonzero_discount')
])
.sort('total_rev', descending=True)
)
print(summary)

```

Note the last expression: `pl.col('discount').filter(pl.col('discount') > 0).mean()` — Polars allows **nested filtering inside an aggregation expression** with no extra GroupBy. This pattern requires a separate `.loc` + `.groupby` chain in Pandas.

Example 3 — Practical: with_columns for feature engineering

python — Code Example

```

# Add multiple derived features in one pass
features = df_pl.with_columns([
    # Arithmetic features
    (pl.col('quantity') * pl.col('unit_price')).alias('gross_value'),
    (pl.col('revenue') / pl.col('quantity')).alias('avg_item_price'),

    # Group-level feature (like Pandas .transform())
    pl.col('revenue').mean().over('category').alias('cat_avg_rev'),
    pl.col('revenue').rank('dense',
descending=True).over('customer_id').alias('customer_rev_rank'),

    # Conditional feature (like np.where)
    pl.when(pl.col('revenue') > 1000)
        .then(pl.lit('High'))
        .when(pl.col('revenue') > 300)
        .then(pl.lit('Medium'))
        .otherwise(pl.lit('Low'))
        .alias('revenue_tier'),

    # String feature
    (pl.col('category') + pl.lit('_') +

```

```
pl.col('region')).alias('segment'),

    # Date feature (if date column exists)
    # pl.col('order_date').dt.month().alias('order_month')
])
print(features.select(['order_id', 'category', 'revenue', 'cat_avg_rev',
                      'customer_rev_rank', 'revenue_tier']).head(5))
```

Student

The `.over()` method looks like Pandas `.transform()` — is that right?

ChatGPT • AI Assistant

Exactly right. `pl.col('revenue').mean().over('category')` computes the mean revenue per category and broadcasts it back to every row — identical in result to Pandas `df.groupby('category')['revenue'].transform('mean')`. The Polars syntax is cleaner because it lives inside `.with_columns()` alongside other feature engineering expressions.

Example 4 — Real Business: Full pipeline side by side

python – Code Example

```
# PANDAS: filter → enrich → groupby → sort
result_pd = (
    df_pd
    .query('is_returned == False')
    .assign(
        profit = lambda d: d['revenue'] - d['quantity'] *
d['unit_price'] * 0.6,
        high_value = lambda d: d['revenue'] > d.groupby('category')
['revenue']
                                                    .transform('mean')
    )
    .groupby('category')
    .agg(
        total_profit = ('profit', 'sum'),
        high_value_orders = ('high_value', 'sum'),
        order_count = ('order_id', 'count')
    )
    .assign(pct_high_value = lambda d: d['high_value_orders'] /
d['order_count'] * 100)
```

```

        .reset_index()
        .sort_values('total_profit', ascending=False)
    )

    # POLARS LAZY: equivalent pipeline
    result_pl = (
        df_pl
        .lazy()
        .filter(pl.col('is_returned') == False)
        .with_columns([
            (pl.col('revenue') - pl.col('quantity') * pl.col('unit_price'))
            * 0.6)
            .alias('profit'),
            (pl.col('revenue') >
pl.col('revenue').mean().over('category'))
            .alias('high_value')
        ])
        .group_by('category')
        .agg([
            pl.col('profit').sum().alias('total_profit'),
            pl.col('high_value').sum().alias('high_value_orders'),
            pl.col('order_id').count().alias('order_count')
        ])
        .with_columns(
            (pl.col('high_value_orders') / pl.col('order_count') * 100)
            .alias('pct_high_value')
        )
        .sort('total_profit', descending=True)
        .collect()
    )

    print('Pandas result:\n', result_pd.head())
    print('Polars result:\n', result_pl.head())
    # Both produce equivalent results

```

❗ COMMON PITFALL / MISTAKE

Incorrect — using Pandas syntax in Polars:

python – Code Example

```

# Pandas habit: df['col']
result = df_pl['revenue'].sum()      # Works in eager mode but
bypasses optimiser
result = df_pl.select(df_pl['revenue'].sum()) # Also works but non-
idiomatic

```

```
# Common error: chaining like Pandas
df_pl['new_col'] = df_pl['revenue'] * 2 # TypeError! Polars
DataFrames are immutable
```

Correct — use expressions inside context methods:

python – Code Example

```
# Always use pl.col() expressions inside
select/with_columns/filter/agg
result = df_pl.select(pl.col('revenue').sum()) # Idiomatic
df_pl = df_pl.with_columns(                    # Returns new
    DataFrame
    (pl.col('revenue') * 2).alias('revenue_x2')
)
```

INTERVIEW CARD

- **Frequency:** Medium and rising — Polars expression API questions appear at companies using it
- **Question:** What is the difference between `.select()` and `.with_columns()` in Polars?
- **Expected Answer:** `.select()` returns only the columns you specify — like SQL `SELECT col1, col2 FROM ...`. `.with_columns()` returns all original columns plus any new or overwritten columns. Use `.select()` when you want to reduce to a subset of columns; use `.with_columns()` for feature engineering where you want to add columns alongside the existing data.
- **Follow-up:** What is `.over()` in Polars and what Pandas operation does it replace?

POLARS WEAKNESSES AND ECOSYSTEM GAPS

ChatGPT • AI Assistant

A quadrant matrix diagram with axes 'Polars strength' (vertical, low to high) and 'Risk if migrating' (horizontal, low to high). Each bubble represents a capability area. Upper-left quadrant (High Polars strength, Low migration risk): 'CSV/Parquet I/O', 'GroupBy aggregation', 'Filter/Select operations', 'Feature engineering' — all labeled green. Upper-right quadrant (High Polars strength, High migration risk): 'Large file performance' — green bubble but with a warning note 'team must learn new API'. Lower-left quadrant (Low Polars strength, Low migration risk): 'Simple transforms on small data' — grey bubble 'Pandas fine here'. Lower-right quadrant (Low Polars strength, High migration risk): four red bubbles: 'scikit-learn integration (needs conversion)', 'MultiIndex operations (no equivalent)', 'Legacy codebase with df.loc heavy usage', 'Excel I/O (partial support)'. In the center, a decision arrow pointing toward the upper-left: 'Sweet spot for Polars: new analytical pipelines, large CSV/Parquet processing, no ML model output'. Purpose: help students visualise which parts of a pipeline are safe to migrate vs which carry real risk.

Polars Ecosystem Gap Map: Where Pandas Still Wins

Student

This all sounds great. Why isn't everyone using Polars already? There must be real downsides.

ChatGPT • AI Assistant

CONCEPT CARD

Polars is genuinely excellent for what it does, but it has four meaningful weaknesses that prevent it from being a universal Pandas replacement today:

1. **ML library friction** — scikit-learn, XGBoost, LightGBM, PyTorch, and TensorFlow all expect NumPy arrays or Pandas DataFrames as input. Polars requires an explicit `.to_pandas()` or `.to_numpy()` conversion at the pipeline boundary.
2. **Smaller community and ecosystem** — Pandas has 10+ years of Stack Overflow answers, tutorials, and library integrations. Polars was first released in 2021 and its community is still building.
3. **Expression API learning curve** — the expression-based model is powerful but requires unlearning Pandas habits. Many Python developers find the first week frustrating.
4. **Incomplete feature coverage** — some Pandas operations have no direct Polars equivalent or behave differently. The index-based model that Pandas supports (`.loc`, `.iloc`, MultiIndex) has no equivalent in Polars.

EXAMPLE CARD

Example 1 — Basic: The ML integration friction

python — Code Example

```
import polars as pl
import numpy as np
```



```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression

# Build features in Polars (fast, idiomatic)
features_pl = (
    df_pl
    .lazy()
    .filter(pl.col('is_returned') == False)
    .with_columns([
        pl.col('revenue').log().alias('log_revenue'),
        pl.col('quantity').cast(pl.Float32).alias('qty_f32'),
        pl.col('discount').fill_null(0.0).alias('discount_clean')
    ])
    .select(['log_revenue', 'qty_f32', 'discount_clean', 'revenue'])
    .collect()
)

# scikit-learn does NOT accept a Polars DataFrame directly
scaler = StandardScaler()
try:
    scaler.fit(features_pl[['log_revenue', 'qty_f32']]) # Would raise
    TypeError
except Exception as e:
    print(f'Error: {e}')

# Correct: convert at the pipeline boundary
features_np = features_pl.select(['log_revenue', 'qty_f32']).to_numpy()
scaler.fit(features_np) # Now works

# Or convert the whole DataFrame to Pandas if needed:
features_pd = features_pl.to_pandas()
X = features_pd[['log_revenue', 'qty_f32', 'discount_clean']].values
y = features_pd['revenue'].values

model = LinearRegression()
model.fit(X, y)
print(f'Model R²: {model.score(X, y):.3f}')
```

Student

Is the `.to_pandas()` conversion expensive?

For most data sizes under a few GB, the conversion is fast — typically under a second — because both Polars (Apache Arrow) and Pandas (NumPy) store numeric data in contiguous memory arrays. Arrow → NumPy is often a zero-copy operation for numeric types. Object/string columns require data copying.

The bigger issue is not speed — it is **the pipeline break**. Once you convert to Pandas, you lose the lazy optimiser for any downstream operations. In a production pipeline, the clean pattern is: **use Polars for all data preparation → convert once at the model boundary → pass NumPy arrays to the ML model**.

Example 2 — Intermediate: Functionality gaps you might encounter

python – Code Example

```
# --- Things that exist in Pandas but have no direct Polars equivalent
---

# 1. MultiIndex – Polars has no index concept at all
# Pandas:
pd_mi = df_pd.set_index(['category', 'region'])
# Polars: you just keep these as regular columns – no equivalent

# 2. .iloc[] positional slicing – Polars has no index
# Pandas: df_pd.iloc[10:20]
# Polars equivalent:
df_pl[10:20]          # Slice syntax works directly
df_pl.slice(10, 10)   # Or explicit slice(offset, length)

# 3. inplace operations – Polars DataFrames are IMMUTABLE
# Pandas:
df_pd.drop(columns=['discount'], inplace=True) # Works (modifies in place)
# Polars: all operations return new DataFrames
df_pl = df_pl.drop('discount') # Must reassign

# 4. SettingWithCopyWarning doesn't exist (Polars is immutable by design)
# You cannot accidentally modify a slice – every operation is a new DataFrame

# 5. pd.read_excel, pd.read_sql – Polars support is partial
# Polars:
pl.read_csv('file.csv')          # Full support
pl.read_parquet('file.parquet')  # Full support, excellent
pl.read_excel('file.xlsx')        # Requires 'xlsx2csv' or 'openpyxl'
```

```
extra
# pl.read_sql(...)           # Available but fewer connector
options
```

Example 3 — Practical: Community and debugging friction

python – Code Example

```
# When you hit an error in Polars, the error messages are often less
helpful
# than Pandas because the community has fewer answered questions

# Example: a common Polars gotcha – duplicate column names in select
try:
    result = df_pl.select([
        pl.col('revenue'),
        pl.col('revenue').mean().alias('revenue') # Duplicate name!
    ])
except pl.exceptions.DuplicateError as e:
    print(f'Polars error: {e}')
    # Must use different alias:
    result = df_pl.select([
        pl.col('revenue'),
        pl.col('revenue').mean().alias('avg_revenue') # Fixed
    ])

# Another gotcha: group_by returns rows in ARBITRARY ORDER
# (Pandas groupby preserves insertion order by default)
# Always add .sort() explicitly if order matters:
result = df_pl.group_by('category').agg(pl.col('revenue').sum())
# May come back as Books, Electronics, Clothing OR any order
result_sorted = result.sort('category') # Explicit sort needed
```

Student

What about when someone on my team who only knows Pandas has to read my Polars code?

ChatGPT • AI Assistant

 REAL WORLD APPLICATION

This is a real operational cost that teams underestimate. A data engineer comfortable with Pandas needs approximately **1–2 weeks** to become fluent in Polars expression syntax. During that transition period, code reviews slow down, debugging is harder, and hiring is more restricted (fewer candidates know Polars). At a startup with a 3-person team this is manageable. At a large organisation with 50 data scientists all used to Pandas, the migration cost is substantial and needs explicit planning.

INTERVIEW CARD

- **Frequency:** High when Polars is on the job description — interviewers want to see balanced judgment
- **Question:** You're evaluating migrating your team's data pipeline from Pandas to Polars. What are the main risks and costs you would evaluate before making the recommendation?
- **Expected Answer:** (1) ML library integration — all scikit-learn/XGBoost pipelines need `.to_pandas()` or `.to_numpy()` boundary conversions. (2) Team learning curve — 1–2 weeks per engineer to become fluent in the expression API. (3) Feature gaps — MultiIndex, inplace operations, some I/O connectors. (4) Community support — fewer Stack Overflow answers and third-party integrations. (5) Downstream code dependencies — any code that expects a Pandas DataFrame breaks. Balance against performance gains: significant for 1–10 GB analytical pipelines, modest for sub-100 MB pipelines.
- **Follow-up:** What is the zero-copy conversion and when does it apply between Polars and NumPy?

DECISION FRAMEWORK: WHEN TO USE PANDAS, WHEN TO USE POLARS

ChatGPT • AI Assistant

A top-down decision flowchart with four diamond decision nodes. Node 1: 'Data > 500 MB OR performance is a documented bottleneck?' — No path leads right to a green terminal box 'Use Pandas — data is small enough, simplest choice'. Yes path continues down. Node 2: 'Is this a new pipeline or explicit rewrite?' — No path leads right to an orange terminal box 'Stay on Pandas — migration cost exceeds benefit for legacy code'. Yes path continues down. Node 3: 'Does output feed an ML library (scikit-learn, XGBoost, etc.)?' — Yes path leads to a yellow terminal box 'Polars + Pandas boundary — use Polars for prep, .to_pandas() once at model input'. No path continues down. Node 4: 'Does your team have or will acquire Polars skills in the next sprint?' — No path leads to an orange terminal box 'Use Pandas + Section 5 optimisations — unowned Polars code is a liability'. Yes path leads to a green terminal box 'Use Polars lazy API — best performance, clean expression model'. On the right side of the chart, an annotation strip: 'Common override situations: If using Parquet I/O → strong push toward Polars. If using Excel I/O → strong push toward Pandas. If joining tables > 1 GB → Polars wins decisively.' Purpose: give students a single printable reference for the migration decision they will encounter in their first data engineering role.

Polars vs Pandas Decision Framework Flowchart

Alright, I have enough context now. Can you give me a concrete framework for when to recommend Polars to my manager?

ChatGPT • AI Assistant

CONCEPT CARD

Here is the decision framework distilled to four questions you answer in order. Stop at the first 'No' — Pandas is the right answer:

1. **Is the data > 500 MB OR is performance a documented bottleneck?** If no, Pandas is fast enough.
2. **Is this a new pipeline or a rewrite?** If no (legacy code), migration cost likely exceeds benefit.
3. **Is the output a file (Parquet/CSV) or another DataFrame?** If the output feeds a scikit-learn model or a library that requires Pandas, the conversion adds friction.
4. **Does your team have or plan to acquire Polars skills?** If no, you own a codebase nobody can debug.

If all four are Yes → Polars. If any are No → Pandas (potentially with the optimisations from Section 5).

EXAMPLE CARD

Example 1 — Basic: The decision function

python — Code Example

```
def recommend_tool(
    data_size_gb,
    is_new_pipeline,
    output_type,          # 'file', 'dataframe', 'ml_model'
    team_knows_polars,
    performance_is_bottleneck
):
    """
    Returns recommended tool and reasoning.
    """
    reasons = []

    # Hard disqualifiers for Polars
    if not is_new_pipeline:
        return 'Pandas', 'Legacy code: migration cost > benefit'
```

```

without full rewrite'

    if output_type == 'ml_model' and not team_knows_polars:
        return 'Pandas', 'ML output + no Polars skills: friction at
every step'

    # Clear Polars wins
    if data_size_gb > 5 and is_new_pipeline:
        reasons.append('Large data benefits from Polars performance')

    if performance_is_bottleneck and is_new_pipeline:
        reasons.append('Documented bottleneck on new pipeline')

    if not reasons:
        return 'Pandas', 'Data small enough, no bottleneck: Pandas
simpler to maintain'

    if output_type == 'ml_model':
        reasons.append('Note: will need .to_pandas() at ML boundary')
        return 'Polars (with Pandas boundary)', ' + '.join(reasons)

    return 'Polars', ' + '.join(reasons)

# Run through real scenarios
scenarios = [
    (0.2, True, 'file', False, False), # Small new ETL
    (8.0, True, 'file', True, True),   # Large new analytics
    pipeline
    (8.0, False, 'file', True, True),  # Large LEGACY pipeline
    (3.0, True, 'ml_model', True, True), # New ML feature
    pipeline
    (0.5, True, 'ml_model', False, False), # Small ML, team new to
Polars
]

for size, new, out, skills, bottleneck in scenarios:
    tool, reason = recommend_tool(size, new, out, skills, bottleneck)
    print(f'{size}GB | new={new} | out={out} | skills={skills} |
bottleneck={bottleneck}')
    print(f' → {tool}: {reason}\n')

```

Example 2 — Intermediate: The practical migration pattern (side by side)

python — Code Example

```

# Real-world pattern for a team migrating incrementally:
# Stage 1 – New analytical pipelines: write in Polars
# Stage 2 – Keep ML model pipelines in Pandas
# Stage 3 – Bridge at the boundary

# STAGE 1: Data preparation in Polars (fast, new code)
def prepare_features_polars(parquet_path: str) → pl.DataFrame:
    return (
        pl.scan_parquet(parquet_path)          # Lazy read from
        Parquet
        .filter(pl.col('is_returned') = False)
        .with_columns([
            pl.col('revenue').log1p().alias('log_revenue'),

            pl.col('revenue').mean().over('category').alias('cat_avg_rev'),
            (pl.col('revenue') /
             pl.col('cat_avg_rev')).alias('rev_vs_cat_avg'),

            pl.col('category').cast(pl.Categorical).alias('category_enc')
        ])
        .select(['order_id', 'log_revenue', 'rev_vs_cat_avg',
            'quantity', 'discount'])
        .collect()
    )

# STAGE 2: Convert at the boundary for ML
def train_model(features_pl: pl.DataFrame):
    # One-time conversion – happens once per training run
    features_pd = features_pl.to_pandas()      # Boundary
    conversion
    X = features_pd.drop(columns=['order_id']).values
    y = np.random.rand(len(features_pd))      # Placeholder
    target

    from sklearn.ensemble import GradientBoostingRegressor
    model = GradientBoostingRegressor(n_estimators=50)
    model.fit(X, y)
    return model

# Compose the pipeline
features =
prepare_features_polars('/tmp/large_orders.csv'.replace('.csv', '.parquet'))

if False else
'/tmp/large_orders.csv')

```

```
# model = train_model(features)
print(f'Features shape: {features.shape}')
print(features.head(3))
```

Student

What about reading Parquet files? I've seen Polars is especially good at that.

ChatGPT • AI Assistant

Polars + Parquet is one of the most powerful combinations in modern data engineering. Parquet is a columnar file format that stores data column by column — exactly matching how Polars reads data. Combined with Polars' `scan_parquet()` (lazy scan, not full load), the query optimiser can push filters and column selections all the way down to the file reader, only reading the bytes it needs:

Example 3 — Practical: Polars lazy scan on Parquet

python — Code Example

```
# Save our dataset as Parquet first
df_pl.write_parquet('/tmp/orders.parquet')

import os
csv_size = os.path.getsize('/tmp/large_orders.csv') / 1024**2
pq_size = os.path.getsize('/tmp/orders.parquet') / 1024**2
print(f'CSV size      : {csv_size:.1f} MB')
print(f'Parquet size: {pq_size:.1f} MB  ({pq_size/csv_size*100:.0f}% of CSV)')
# CSV size      : 87.4 MB
# Parquet size: 24.1 MB  (28% of CSV) — Parquet compression

# scan_parquet — lazy, projection and filter pushdown all the way to
disk
result = (
    pl.scan_parquet('/tmp/orders.parquet')
    .filter(pl.col('category') == 'Electronics') # pushed to file
reader!
    .select(['order_id', 'revenue', 'quantity']) # only 3 cols read
from disk
    .group_by('order_id')
    .agg(pl.col('revenue').sum())
    .sort('revenue', descending=True)
```



```

        .limit(10)
        .collect()
    )
    print(result)
    # Polars reads ONLY the Electronics rows from ONLY 3 columns
    # – even though the full file has 10 columns and 1M rows

```

Example 4 — Real Business: Complete benchmark across the full pipeline

python – Code Example

```

import time

def pandas_pipeline(filepath):
    df = pd.read_csv(filepath,
                      dtype={'customer_id': 'int32', 'quantity': 'int8',
                             'unit_price': 'float32', 'revenue': 'float32'})
    return (
        df[df['is_returned'] == False]
        .assign(log_rev = lambda d: np.log1p(d['revenue']),
                cat_avg = lambda d: d.groupby('category')['revenue']
                                      .transform('mean'))
        .groupby('category')
        .agg(total=('revenue', 'sum'), mean_log=('log_rev', 'mean'),
              orders=('order_id', 'count'))
        .reset_index()
        .sort_values('total', ascending=False)
    )

def polars_pipeline(filepath):
    return (
        pl.scan_csv(filepath)
        .filter(pl.col('is_returned') == False)
        .with_columns([
            pl.col('revenue').log1p().alias('log_rev'),
            pl.col('revenue').mean().over('category').alias('cat_avg')
        ])
        .group_by('category')
        .agg([
            pl.col('revenue').sum().alias('total'),
            pl.col('log_rev').mean().alias('mean_log'),
            pl.col('order_id').count().alias('orders')
        ])
        .sort('total', descending=True)
        .collect()
    )

```

```

    )

start = time.perf_counter()
r_pd = pandas_pipeline('/tmp/large_orders.csv')
t_pd = time.perf_counter() - start

start = time.perf_counter()
r_pl = polars_pipeline('/tmp/large_orders.csv')
t_pl = time.perf_counter() - start

print(f'Pandas pipeline : {t_pd:.3f}s')
print(f'Polars pipeline : {t_pl:.3f}s')
print(f'Speedup          : {t_pd/t_pl:.1f}x')
# Pandas pipeline : 3.871s
# Polars pipeline : 0.412s
# Speedup          : 9.4x

```

❗ COMMON PITFALL / MISTAKE

Incorrect — migrating a legacy Pandas pipeline piece by piece without a clear boundary:

python – Code Example

```

# Anti-pattern: converting back and forth multiple times
df_pl = pl.from_pandas(df_pd)           # Convert to Polars
result = df_pl.group_by('cat').agg( ... ) # Process in Polars
df_pd2 = result.to_pandas()             # Back to Pandas for one
operation
df_pl2 = pl.from_pandas(df_pd2)         # Back to Polars
# Each round-trip costs time and breaks the lazy graph

```

Correct — single boundary, single direction:

python – Code Example

```

# One clean boundary: Polars for all data prep, Pandas only at the end
result_pl = (
    pl.scan_parquet('data.parquet')
    .filter( ... ).with_columns( ... ).group_by( ... ).agg( ... )
    .collect()
) # All in Polars
result_pd = result_pl.to_pandas() # One conversion, only if
downstream needs it

```

🔮 AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

"Act as a senior data engineer who uses both Pandas and Polars in production. I want to learn Polars by rewriting Pandas pipelines. Give me 10 exercises where each exercise: (1) shows a Pandas pipeline of 3–5 steps on a retail dataset, (2) asks me to rewrite it in Polars lazy mode using the expression API, (3) after I attempt it shows the optimal Polars solution, (4) points out any API differences I missed (*groupby vs groupby, descending vs ascending, pl.when vs np.where*), (5) shows the performance difference, (6) notes any Polars-specific optimisation I could add (*scanparquet, predicate pushdown, streaming mode*). Start simple (filter + select) and build to joins, window functions, and time series resampling. Do not reveal the solution until I attempt it."

🔥 EXERCISE

Rewrite both of these Pandas pipelines in Polars lazy mode:

1. ```python

```
result = (
    dfpd[dfpd['quantity'] >= 3]
    .assign(effectiveprice = lambda d: d['unitprice'] * (1 - d['discount']))
    .groupby(['category','region'])
    .agg(avgeffectiveprice=('effective_price','mean'),
        totalorders=('orderid','count'))
    .reset_index()
    .sort_values('avgeffective_price', ascending=False)
)
```

code – Code Example

2. ```python

```
# Find the top 3 customers by revenue in each category
top3 = (
    df_pd
    .groupby(['category','customer_id'])['revenue'].sum()
    .reset_index()
    .sort_values(['category','revenue'], ascending=[True,False])
)
```

```
.groupby('category').head(3)
)
```

SUMMARY CARD

Section 6 Key Takeaways:

- Polars is 5–20× faster than Pandas on complex analytical pipelines due to Rust, no GIL, SIMD, and lazy query optimisation
- The **lazy API** (`.lazy()` + `.collect()`) is Polars' most powerful feature — it enables filter pushdown, projection pushdown, and automatic parallelism
- Core Polars expression API: `pl.col()` builds expressions; `.select()` picks columns; `.with_columns()` adds columns; `.filter()` filters rows; `.group_by().agg()` aggregates; `.over()` does group-level transforms
- `.alias()` renames expression output — equivalent to SQL `AS`
- `pl.when().then().otherwise()` replaces `np.where` and multi-condition `np.select`
- `.over('group_col')` replaces Pandas `.groupby().transform()`
- Polars DataFrames are **immutable** — every operation returns a new DataFrame; `inplace=True` does not exist
- `group_by` returns rows in **arbitrary order** — always `.sort()` if order matters
- Polars weaknesses: scikit-learn/ML friction (`.to_pandas()` needed), no MultiIndex, smaller community, 1–2 week learning curve per engineer
- Decision rule: Polars for new pipelines > 500 MB with file/DataFrame output and a team willing to learn; Pandas for legacy code, small data, or ML model output with no conversion budget
- **Polars + Parquet** is the highest-performance combination: `pl.scan_parquet()` with filter/projection pushdown reads only the bytes needed from disk

🔗 PRACTICE QUIZ

1. What must you call to execute a Polars lazy pipeline?
2. What is the Polars equivalent of Pandas `df.groupby('c')['a'].transform('mean')` ?
3. What does `pl.when(cond).then(A).otherwise(B)` replace in Pandas/NumPy?
4. Predict: `df_pl.group_by('category').agg(pl.col('revenue').sum())` — is the output sorted by category?
5. What file format gives the best performance with Polars and why?

6. Debug: `df_pl['new_col'] = df_pl['revenue'] * 2` — what error does this raise and how do you fix it?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. `.collect()` — without it, the pipeline returns a `LazyFrame` object (just the query plan)
2. `pl.col('a').mean().over('c')` inside `.with_columns()`
3. `np.where(cond, A, B)` — or for multi-condition: `np.select(conditions, choices, default)`
4. No — `group_by` in Polars returns rows in arbitrary order; add `.sort('category')` explicitly
5. Parquet — columnar format matches Polars' columnar memory model; `scan_parquet()` enables filter and projection pushdown to the file reader so Polars reads only needed bytes
6. `TypeError: 'DataFrame' object does not support item assignment` — Polars DataFrames are immutable; fix: `df_pl = df_pl.with_columns((pl.col('revenue') * 2).alias('new_col'))`

❓ **CHECKPOINT QUESTION**

Your team has a 6 GB daily CSV pipeline that: filters to active orders, computes log revenue and a category-level z-score, joins with a 10K-row products table, and outputs to Parquet. The current Pandas pipeline takes 4 minutes. List every Polars optimisation that would apply to this specific pipeline and estimate the total speedup.

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including pandas vs polars, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.

- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Pandas vs Polars Architecture: Eager vs Lazy Execution:** A two-column architecture comparison diagram. Left column labeled 'PANDAS — Eage...